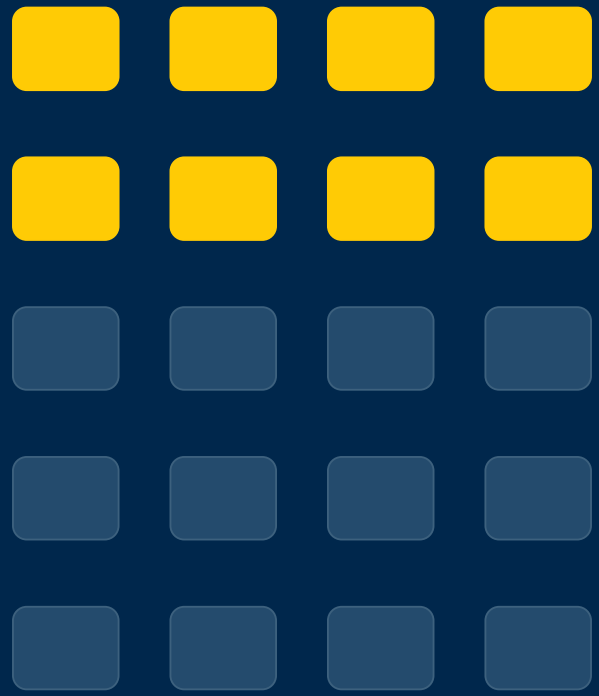
SLURM:

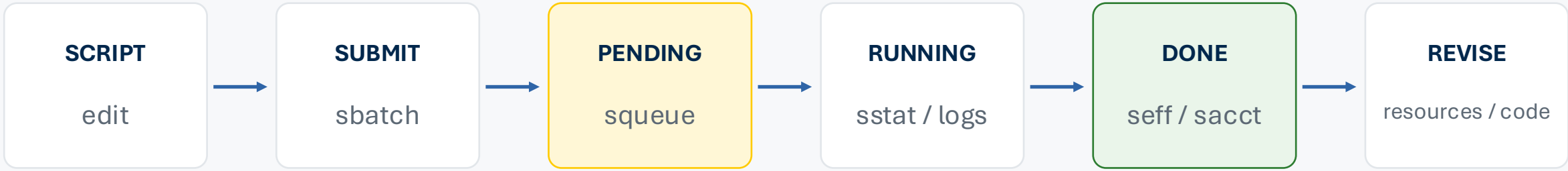
From a Single Job to Efficient Parallel Computing



Resource right-sizing → accounting → job arrays → reliable fan-in

Fritsche Lab · University of Michigan · September 2026

90-second refresher: the job lifecycle




```
sbatch job.sbatch
squeue -u "$USER"
scontrol show job JOB_ID
scancel JOB_ID
seff JOB_ID
sacct -j JOB_ID ...
```

The scheduler is not the debugger

State and pending reason tell you where to look.
The application log and exit code tell you what the process actually did.

MENTAL MODEL

One task, many CPUs, or many tasks?

Serial program

One process
One useful CPU

Request:
--ntasks=1
--cpus-per-task=1

Threaded program

One process
Multiple threads

Request:
--ntasks=1
--cpus-per-task=N

Configure N threads.

Independent repetitions

Many one-task jobs
Phenotypes, chromosomes, files, simulations

Use a job array.

Allocating CPUs does not make serial code parallel.

Anatomy of a robust single-node job

```
#!/usr/bin/env bash
#SBATCH --job-name=analysis
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G
#SBATCH --time=00:05:00
#SBATCH --output=logs/%x_%j.out
#SBATCH --error=logs/%x_%j.err

set -Eeuo pipefail
source scripts/runtime_setup.sh
print_job_context
run_analysis_task 1 results/task_001.csv
```

Explicit envelope
CPU, memory, and time are reviewable hypotheses.

Unique logs
%x and %j prevent collisions and preserve evidence.

Fail loudly
set -Eeuo pipefail avoids a false “COMPLETED”.

Reproducible runtime
Load modules / environments inside the batch shell.

Visible context
Record host, job IDs, executable, and timestamps.

Request a defensible resource envelope

CPU

Can the program use threads or child processes?

Request one CPU for a serial worker. Match thread settings to SLURM_CPUS_PER_TASK.

MEMORY

What did representative jobs actually peak at?

Use MaxRSS across multiple tasks. Add a safety margin, not an order of magnitude.

TIME

What is a credible upper-tail runtime?

Shorter accurate limits schedule more easily. Split or checkpoint long work.

A resource request is a hypothesis. Measure it, then revise it.

Current U-M defaults make explicit requests important

SETTING	GREAT LAKES	ARMIS2
Default walltime	60 min	60 min
Default memory per CPU	768 MB	768 MB
Standard max walltime	14 days	14 days
Max queued jobs / user / account	5,000	5,000
Login-node envelope	2 CPUs / 4 GB	2 CPUs / 4 GB
Scratch	10 TB + 1M inodes	10 TB
Scratch lifecycle	60-day access purge; no backup	60-day access purge; no backup

Great Lakes: HIPAA/PHI is not permitted.

Armis2: sensitive/HIPAA work with approved storage.

Sources: U-M ARC Great Lakes and Armis2 defaults/policies. Account or association limits may be stricter.

ILLUSTRATIVE OUTPUT

seff: fast diagnosis of one completed job

```
Job ID: 48123001
State: COMPLETED (exit code 0)
Cores: 4
CPU Utilized: 00:00:42
CPU Efficiency: 24.1%
Job Wall-clock time: 00:00:43
Memory Utilized: 118.4 MB
Memory Efficiency: 1.4% of 8.00 GB
```

CPU

Approximately one CPU was used although four were allocated.

Memory

Observed peak is far below 8 GB. Confirm across representative tasks before reducing.

Action

Request one CPU and a smaller memory envelope; keep a margin and re-measure.

Low CPU efficiency can also mean I/O wait — inspect context before “fixing” it.

sacct: compare the whole array

```
sacct -j "$JOB_ID" --units=G \
  -o JobIDRaw,State,Elapsed,Timelimit,\
     AllocCPUS,ReqMem,MaxRSS,TotalCPU,ExitCode
```

TASK	STATE	ELAPSED	CPU	MAXRSS
48123110_1	COMPLETED	00:00:13	1	0.11G
48123110_2	COMPLETED	00:00:16	1	0.12G
48123110_3	COMPLETED	00:00:11	1	0.11G

Why MaxRSS may look blank

For some Slurm accounting layouts, memory appears on the .batch step rather than the allocation row.

Look at distributions

Median and upper-tail runtime
Largest MaxRSS
Failed states / exit codes
Task-size outliers

One seff report is a clue. Array-wide sacct is evidence.

EXAMPLE

24 phenotype models

COHORT

N

32,000–52,000 participants per task

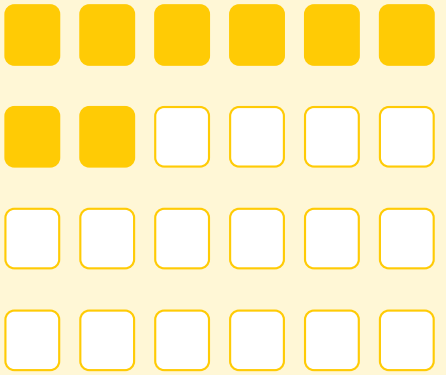
Predictor + age + sex + PCs
Binary and quantitative outcomes

ONE WORKER

```
analyze_one_trait.py
  --task-id 6
  --output
task_006.csv
```

Reads one manifest row
Generates deterministic data
Fits one model
Writes one result

24 INDEPENDENT ROWS



Pain, fatigue, mental health, medication exposure, and common diagnoses

ONE ALLOCATION, 24 MODELS

The serial anti-pattern

No concurrency

Independent work waits in line.

Coarse recovery

A late failure can force a large rerun.

One envelope

Every model receives resources sized for the largest.

Weak provenance

Logs and outputs are harder to map to one work unit.

Approximate elapsed: 24 × task runtime

ONE STABLE ROW = ONE TASK

Put work units in a manifest

```
task_id  phenotype            type         n      seed  work
1        pain_burden          continuous   42000  1101  5
2        fatigue_score        continuous   38000  1102  5
...
6        fibromyalgia_code    binary       48000  1106  7
...
24       diabetes_code        binary       50000  1124  6
```

AUDITABLE

Inputs, seeds, and task sizes are explicit.

STABLE

Task 6 always means the same work unit.

SUBSETTABLE

Rerun 6,11,19 without renaming files.

EXTENSIBLE

Add resource class or chunk fields later.

Avoid using directory order as an implicit scheduler API.

Make the worker responsible for exactly one row

```
./analysis/analyze_one_trait.py \
  --manifest data/manifest.tsv \
  --task-id 6 \
  --output results/local/task_006.csv
```

Local smoke test

./scripts/run_local.sh 3

Prove correctness before involving the scheduler.

Validate task ID → Read manifest row → Run one model → Write one CSV → Exit non-zero on failure

Parallelizing a broken worker creates 24 broken jobs faster.

Convert the worker into an array element

```
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G
#SBATCH --time=00:05:00
#SBATCH --output=logs/%x_%A_%a.out
#SBATCH --error=logs/%x_%A_%a.err
#SBATCH --array=1-24

TASK_ID="${SLURM_ARRAY_TASK_ID:?}"
printf -v OUTPUT "results/%s/task_%03d.csv" \
  "$SLURM_ARRAY_JOB_ID" "$TASK_ID"
run_analysis_task "$TASK_ID" "$OUTPUT"
```

SLURM_ARRAY_TASK_ID

Selects the manifest row.

%A and %a

Unique array/job and task IDs in log names.

Same initial envelope

Split heterogeneous tasks into resource classes.

SchedMD: arrays share initial options; %x is the job name; %A/%a expand to array job ID and task index.

1–24%8

Submit dynamically and cap concurrency

```
N=$(( $(wc -l < data/manifest.tsv) - 1 ))
ARRAY_SPEC="1-${N}%${MAX_CONCURRENT}"

array_job=$(sbatch --parsable \
  --account="$SLURM_ACCOUNT" \
  --partition="$SLURM_PARTITION" \
  --array="$ARRAY_SPEC" \
  slurm/array.sbatch)
```

Why throttle?

Protect account limits and shared storage.
Avoid turning a clean array into an I/O storm.

Do not hard-code a lab account into a reusable batch file.

SchedMD: "%" sets the maximum number of simultaneously running array tasks.

LIVE TERMINAL

Monitor the array as one workflow and many tasks

```
squeue -j "$ARRAY_JOB_ID"
squeue -r -j "$ARRAY_JOB_ID"
./scripts/monitor.sh "$ARRAY_JOB_ID"

tail -f
logs/slurm_array_${ARRAY_JOB_ID}_6.out
./scripts/sacct_summary.sh "$ARRAY_JOB_ID"
```

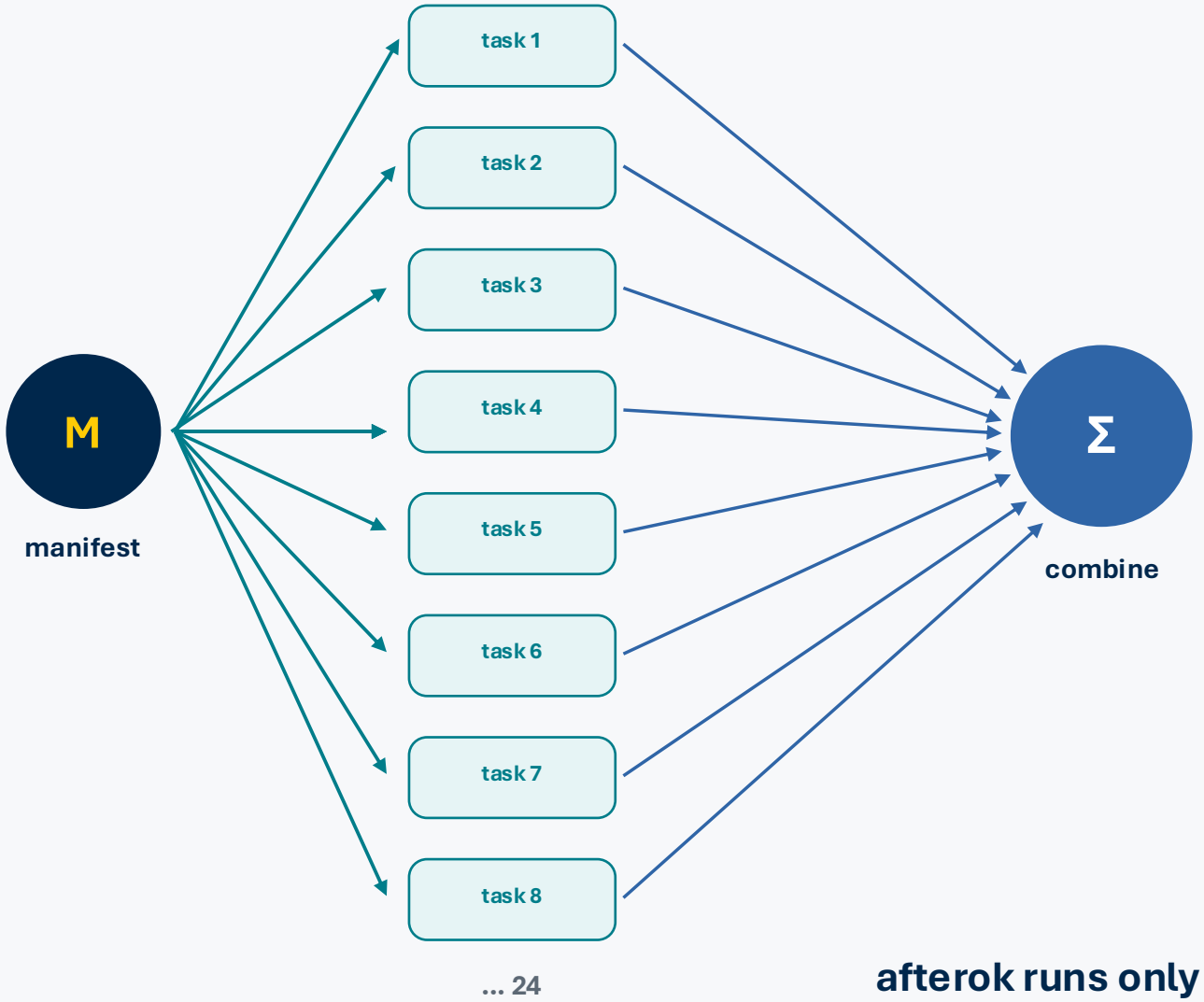
```
48123110_1          R   gl3102
48123110_2          R   gl3110
48123110_3          R   gl3114
48123110_4          R   gl3121
48123110_[9-24%4] PD  (JobArrayTaskLimit)
```

Read the reason

JobArrayTaskLimit is expected: the concurrency cap is doing its job.

Each element has its own state, host, log, exit code, and result file.

Fan out, then fan in with a dependency



combined_results.csv

summary.md
completeness check
FDR correction

```
--dependency="afterok:${array_job}"
```

afterok runs only after every array element succeeds.

Rerun only failed elements

3 failed

```
./scripts/rerun_failed.sh 48123110

# prints:
./scripts/submit_pipeline.sh \
  --tasks "6,11,19" \
  --results-dir "results/48123110" \
  --max-concurrent 3
```

Preserve successful work

One output per task allows localized recovery without deleting valid results.

Diagnose first

Open each failed task's state, exit code, and element-specific log before resubmitting.

When one array envelope no longer fits

Option 1: resource classes

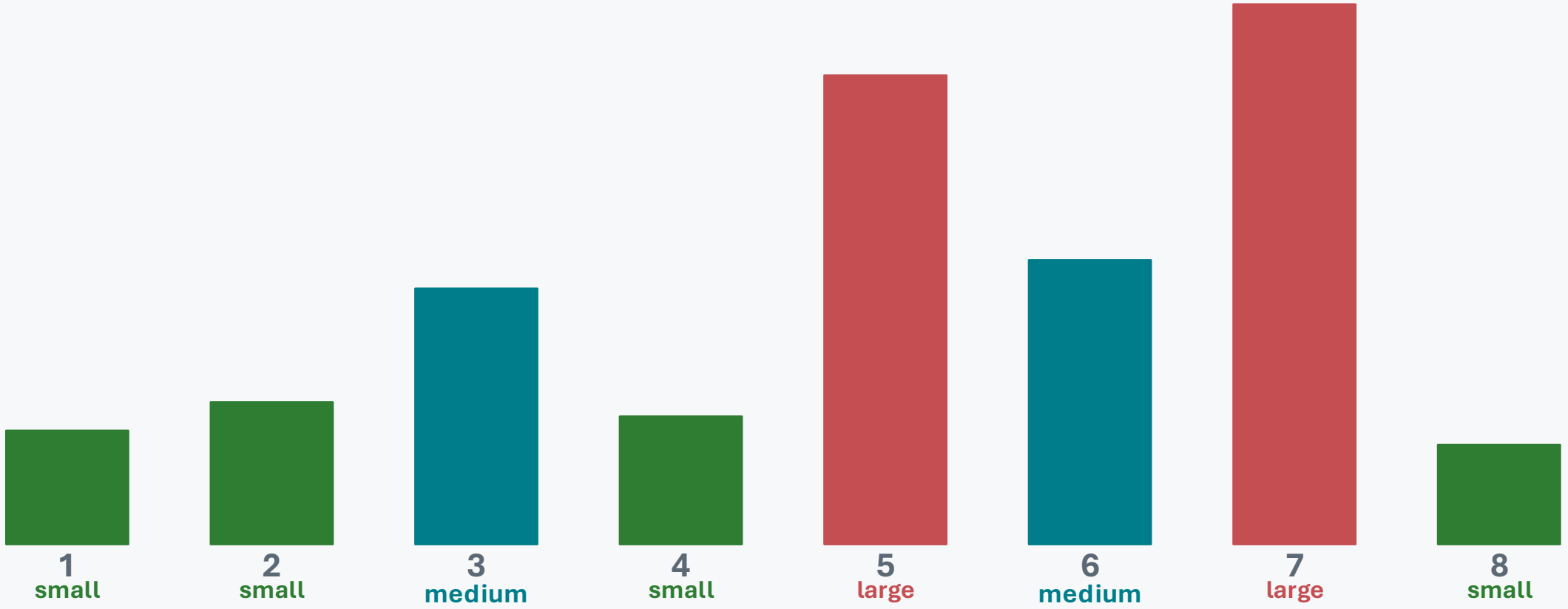
Split the manifest into small / medium / large arrays.

Option 2: finer chunks

Reduce the largest work units so their tails are less extreme.

Option 3: workflow engine

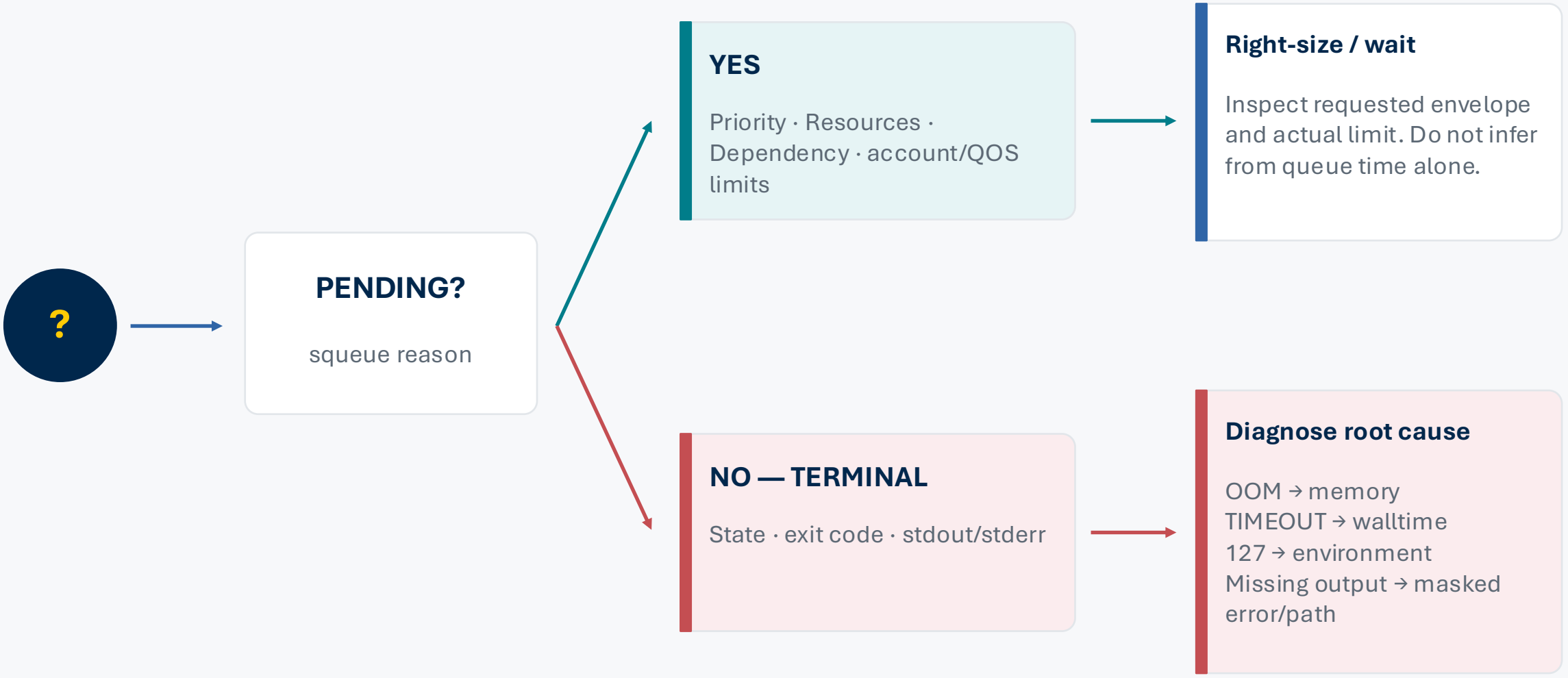
Express per-process or dynamic resource rules.



Do not size every task for the single largest outlier.

DECISION TREE

Troubleshooting: start with state and reason



STATE / REASON → LOGS → EXIT CODE → RESOURCES → ENVIRONMENT → I/O

OPERATIONAL CHECKLIST

Great Lakes / Armis2 problems that recur

Login-node computation

2 CPUs / 4 GB; Armis2 processes also have a short CPU-time limit. Use salloc, debug, or Open OnDemand.

Wrong account or partition

Set and export SLURM_ACCOUNT and SLURM_PARTITION before submission.

Scratch treated as storage

Temporary, no backup, 60-day access purge.
Move durable outputs elsewhere.

Array log directory missing

Create logs/ before sbatch; Slurm will not create parent directories.

Environment differs in batch

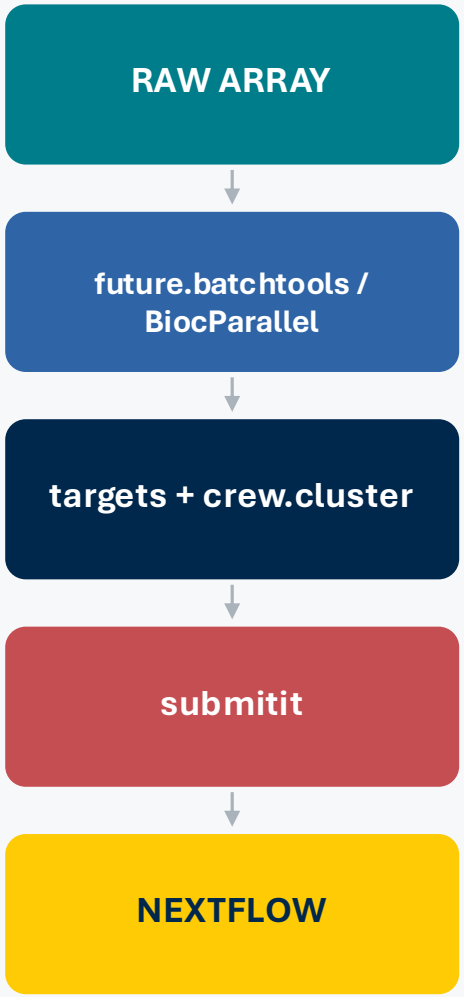
Initialize modules / Conda / R libraries inside the script and print versions.

Shared-filesystem I/O storm

Throttle arrays, stage data, reduce tiny files, and avoid simultaneous reads of one huge input.

Data boundary: Great Lakes ≠ PHI. Armis2 still requires approved storage and handling.

Use the smallest abstraction that removes real complexity



Homogeneous independent tasks	Transparent, portable, easiest to debug
Existing R parallel API	Scheduler backend with modest code change
R dependency graph + caching	Runs only outdated targets; SLURM workers
Python functions	Local/Slurm execution and array mapping
Multi-stage file pipeline	Containers, resume, reports, per-process resources

Every wrapper still relies on correct CPU, memory, time, account, partition, and failure handling.

CLOSE

Six rules to take back to your own analyses

1. Prove one task locally.
2. Request only CPUs the code can use.
3. Tune from seff/sacct distributions.
4. Represent work in a stable manifest.
5. Throttle arrays and use unique logs.
6. Aggregate after success; rerun only failures.

Efficient SLURM is measurable, modular, and recoverable.

Appendix: command reference

```
# Submit
TASK_ID=3 sbatch --export=ALL,TASK_ID \
  slurm/single.sbatch
./scripts/submit_array.sh
# Monitor
squeue -j JOB_ID
squeue -r -j ARRAY_JOB_ID
./scripts/monitor.sh ARRAY_JOB_ID

# Inspect
seff JOB_ID
./scripts/sacct_summary.sh JOB_ID
scontrol show job JOB_ID

# Recover
./scripts/rerun_failed.sh ARRAY_JOB_ID
scancel ARRAY_JOB_ID_6
```

```
# Common pending reasons
Priority
Resources
Dependency
JobArrayTaskLimit
AssocMaxJobsLimit
QOS...

# Common terminal states
COMPLETED
FAILED
OUT_OF_MEMORY
TIMEOUT
CANCELLED
NODE_FAIL
```